DEVOPS REFERENCE

AIDEVSECOPS · CLOUD ENGINEERING

LLM Inference Infrastructure

What actually happens between you pressing Enter and the model streaming a reply — from GPU silicon up through Kubernetes, and every DevOps concept that lives in between.

Purpose: AIDevSecOps focus

Scope: GPU · vLLM · llm-d · Kubernetes · Helm · Gateway API · Observability

Contents

01 The Hardware Layer

> Every abstraction above — Kubernetes objects, Helm charts, routing algorithms — exists to feed the correct data to the correct piece of silicon at the correct time. Understanding the physical substrate first makes every layer above it easier to reason about.

The Five Components

Component	What it is	Role in inference
CPU	General-purpose processor, few high-clock cores, optimized for branching and control flow.	OS, networking, request deserialization, tokenization, launching GPU kernels. Does not run the model.
RAM	System memory, large capacity (512GB–2TB per server), moderate bandwidth (~200–400 GB/s DDR5).	Staging buffer for model weights before GPU transfer, OS and process memory.
GPU	Thousands of small cores optimized for identical parallel operations — matrix multiplication at scale.	The actual model forward pass. Both prefill and decode arithmetic run here.
VRAM (HBM)	Memory physically on the GPU package. Small capacity (80–192GB) but very high bandwidth (~3.3 TB/s HBM3).	Holds model weights and KV cache. This is usually the real bottleneck, not compute.
Bandwidth	Rate of data movement. Different at every link in the stack.	Determines how fast the GPU can be fed. See table below.

Bandwidth Is Not One Number

Link	Bandwidth	Notes
GPU ↔ VRAM (HBM3)	~3,300 GB/s	Fastest link on the whole node. Everything the GPU needs should live here during inference.
GPU ↔ GPU (NVLink)	~900 GB/s	Enables model sharding across GPUs on one node.
Node ↔ Node (InfiniBand)	~400 Gbps	Multi-node clusters. Required for disaggregated serving over RDMA.
CPU/RAM ↔ GPU (PCIe Gen5 x16)	~64 GB/s	The narrowest link. Data crosses this only at request boundaries.

Compute vs. Memory — The Distinction That Drives Every Design Decision

Every workload is either **compute-bound** (limited by arithmetic throughput, measured in FLOPS) or **memory-bandwidth-bound** (limited by how fast data can be fed to the cores). This distinction is the single most important concept in inference infrastructure design.

> **WHY THIS MATTERS DOWNSTREAM**
>
> Prefill is compute-bound. Decode is memory-bandwidth-bound. This is not a nuance — it is the reason vLLM has a scheduler, the reason llm-d exists, the reason disaggregated serving is worth its network cost, and the reason GPU node pools get provisioned differently for different roles.

02 Tokens — How Your Prompt Becomes Numbers

> The model has no concept of letters or words internally. It operates on sequences of integers drawn from a fixed vocabulary. Everything upstream is preprocessing to produce that integer sequence; everything downstream operates purely on it.

What a Token Is

A token is a **subword unit** from a fixed vocabulary the model was trained with. Vocabularies typically hold 30,000–130,000 entries, each mapped to a unique integer ID. Common words become one token; rare or compound words split into multiple pieces; arbitrary binary or emoji input still tokenizes because most modern tokenizers operate on raw UTF-8 bytes.

The Six-Stage Pipeline

1. **Raw prompt.** A sequence of Unicode characters stored as UTF-8 bytes. Newlines are ordinary characters at this point.
2. **Normalization.** Some tokenizers apply Unicode NFC/NFKC normalization. Modern byte-level BPE tokenizers skip this and operate directly on raw bytes.
3. **Pre-tokenization.** Text is split into rough chunks on whitespace and punctuation boundaries via regex.
4. **Subword merging (BPE).** During training, the most frequent adjacent token pairs were repeatedly merged into new tokens, producing a ranked list of merge rules. At inference, those merges are applied in order to the pre-tokenized text.
5. **Vocabulary lookup.** Each resulting subword piece is replaced with its integer ID — a simple hash-map lookup.
6. **Token ID sequence.** A flat list of integers is what actually reaches the model's embedding layer.

Operational Consequences

- **Rate limits are token-based, not request-based.** An API gateway meters TPM (tokens-per-minute), not RPM alone, because tokens drive compute cost.
- **Context windows are token counts.** A "200K context window" means 200,000 tokens. English averages ~4 characters per token; code and non-English text tokenize less efficiently.

- **Special tokens are injected before your text.** Multi-turn chat wraps your content in reserved role markers (`<|user|>` , `<|assistant|>`) the model was specifically trained to recognize.
- **Tokenization runs on CPU**, before the GPU ever sees the request.

03 The Two Halves of Inference

> When you press Enter, you experience two distinct things: a pause of one or two seconds, then a stream of words. These are two completely different phases of computation running on the same GPU, bottlenecked on opposite resources.

The Pause — Prefill

Everything before the first token appears is bundled into **time-to-first-token (TTFT)**. It includes network round-trip, gateway auth and rate-limit checks, safety classification, tokenization, scheduler queueing, and the prefill forward pass itself.

Prefill is **compute-bound and embarrassingly parallel**. All prompt tokens go through the model's layers simultaneously in one large matrix multiplication — token 5 does not need token 4's output to be computed first. This is why prefill is fast per-token but total duration still scales with prompt length: a 50-token prompt prefills almost instantly; a 50,000-token prompt takes noticeably longer because it is still one (much larger) matrix operation.

The Stream — Decode

Decode is **autoregressive**: generating token N+1 requires token N to already exist, because the model feeds its own output back in as input for the next step. There is no way to compute tokens 6 and 7 in parallel before token 5 is known — the dependency is genuine.

Each forward pass reads the model's weights and the growing KV cache from VRAM, performs relatively little arithmetic, and produces one token. Cores sit mostly idle waiting on memory bandwidth. This is why decode is **memory-bandwidth-bound**, measured as **time-per-output-token (TPOT)**.

Side by Side

Property	Prefill (pause)	Decode (stream)
Bottleneck	Compute (FLOPS)	Memory bandwidth (VRAM ↔ GPU)
Parallelism	All prompt tokens at once	Strictly one token at a time
Scales with	Prompt length	Output length
Metric	TTFT	TPOT / inter-token latency
Regression cause	Scheduler queueing, safety-classifier slowdown	VRAM/HBM bandwidth contention from over-batching

SPECULATIVE DECODING

The one partial exception to strict sequentiality. A small draft model guesses several tokens ahead, and the main model verifies them in one batched pass. When guesses are correct, effective tokens-per-step exceeds 1, letting providers stream faster than raw memory-bandwidth math would suggest.

04 vLLM — The Single-Node Serving Engine

> vLLM is the software layer sitting directly on the GPU that turns raw compute into a production-viable serving engine. It solves single-node efficiency; it does not solve cluster-wide concerns.

The Problems vLLM Solves

KV cache fragmentation

Naive serving pre-allocated contiguous GPU memory for each request's maximum possible sequence length. Padding overhead ran 60–80% for typical workloads, and GPU SM utilization sat around 30–40% even at moderate load.

Static batching

Traditional batching waited for every sequence in a batch to finish before accepting new work. If one request generated 1,000 tokens while others generated 10, the GPU idled waiting on the long one.

The Two Core Fixes

PAGEDATTENTION

Maps each sequence's KV cache through a logical block table to non-contiguous physical blocks in GPU memory, applying virtual-memory-style paging. Eliminates internal fragmentation and enables prefix sharing — a shared system prompt is computed and stored once, then reused across many concurrent users.

CONTINUOUS BATCHING

The scheduler checks the request queue at every decode step. When a request finishes, its KV cache blocks are freed and a new request is slotted in immediately. The GPU never idles waiting on the slowest sequence in a batch.

Combined, these produce roughly 2–4× throughput over naive serving, which is why vLLM is the most widely adopted open-source inference engine for production LLM deployments.

The Full Production Feature Set

- **Quantization** — GPTQ, AWQ, INT4, INT8, FP8. Runs the model at lower numerical precision to fit more in VRAM.
- **Speculative decoding** — Small draft model guesses ahead; main model verifies in a single pass.

- **Multi-LoRA support** — Serve many fine-tuned adapters on one base model, so 50 customers with 50 fine-tunes share one GPU fleet.
- **Tensor and pipeline parallelism** — Shard models too large for one GPU across multiple devices.
- **Prefix caching and chunked prefill** — Further optimizations building on PagedAttention.
- **OpenAI-compatible API server** — Drop-in replacement for the OpenAI API; existing tooling works with a base-URL change.
- **Broad hardware support** — NVIDIA, AMD, Intel, TPU, ARM CPUs. Avoids vendor lock-in.

05 How One GPU Serves Many Users at Once

> The economics of GPU inference depend on this mechanism. A GPU serving one user at a time wastes almost all its memory bandwidth. Batching converts a memory-bound operation into one with much higher arithmetic intensity.

The Core Insight

Decode is memory-bandwidth-bound because each forward pass reads the model's weights from VRAM and performs relatively little arithmetic on them. Serving one user pays that full weight-read cost to compute one token.

Batching multiplies the same weight matrix against a **batch of activation vectors** — one per user — in a single pass. The weights are read from VRAM exactly once, and that single expensive read produces next-tokens for many users simultaneously. This is why increasing batch size increases *total* throughput without necessarily making any single user's response faster.

What Limits Batch Size

Constraint	What it means operationally
VRAM for KV cache	Each concurrent user needs their KV cache resident in VRAM. Formula: `available_VRAM = total_VRAM - model_weights`. PagedAttention lets you pack far more caches into the same budget than naive allocation would.
Prefix caching benefit	Shared system prompts and few-shot examples are computed and stored once, dramatically raising max concurrent users per unit of VRAM.
Compute ceiling	Beyond a batch-size threshold, the GPU crosses back into compute-bound territory and adding more users stops being free.

Chunked Prefill — Mixing Prefill and Decode Safely

New users constantly arrive and need prefill while existing users are mid-decode. A large prefill can monopolize a full GPU step and stall everyone's decode. Chunked prefill splits large prefills into smaller pieces interleaved with ongoing decode steps in the same batch, keeping TPOT predictable for existing users even while new ones join.

When Not To Share — MIG

For regulated tenants requiring guaranteed isolated performance, NVIDIA Multi-Instance GPU (MIG) physically partitions a single GPU into fully isolated instances with dedicated compute and VRAM slices. You sacrifice shared-batching throughput to gain hard tenant isolation — a real architectural trade-off decided at the Terraform / node-pool provisioning layer.

06 llm-d — Cluster-Level Orchestration on Kubernetes

> Everything vLLM does happens inside a single GPU replica. Real production has many replicas, and the moment you have many replicas, you have four new problems that batching alone cannot solve.

The Four Problems llm-d Solves

1. **Routing** — Which replica should this new request go to?
2. **Cache locality** — If a user sends a follow-up, which replica already has their conversation's KV cache warm?
3. **Bottleneck mismatch** — Prefill and decode have opposite resource needs. Should they even run on the same replicas?
4. **Independent scaling** — How do you scale each side of that mismatch on its own signal?

The Four Answers

CACHE-AWARE ROUTING

Extends vLLM's per-GPU prefix caching across the whole fleet. The scheduler tracks which replica already holds which conversation's KV cache and routes continuation requests back to that specific replica, avoiding full recomputation.

DISAGGREGATED SERVING

Deploys prefill and decode as separate pod pools. A burst of new conversations scales the prefill pool; a burst of long ongoing generations scales the decode pool. Neither over-provisions to cover the other.

GPU-AWARE, METRIC-DRIVEN SCHEDULING

Instead of round-robin, the scheduler uses real signals from vLLM's Prometheus endpoint — KV cache utilization, queue depth, batch occupancy — to make routing decisions. Domain-aware, not generic.

RIGHT-METRIC AUTOSCALING

Prefill pools scale on request rate and compute saturation. Decode pools scale on active KV-cache occupancy and per-token latency drift. Standard HPA/KEDA pattern, GPU-serving-specific metrics.

Where It Came From

llm-d is a CNCF sandbox project founded by Red Hat, Google Cloud, IBM Research, CoreWeave, and NVIDIA, accepted into CNCF on 24 March 2026. This governance matters in build-vs-vendor discussions: llm-d is open and vendor-neutral, while alternatives like NVIDIA Dynamo are proprietary orchestration layers sitting above vLLM outside Kubernetes.

07 How Requests Are Routed — Filter · Score · Pick

> A single request never goes to two replicas. It goes through a decision pipeline that filters candidates, scores survivors, and picks exactly one. This logic lives in the Endpoint Picker (EPP), a Kubernetes Deployment sitting between the gateway and the vLLM pods.

Why Round-Robin Fails Here

Standard load balancing distributes traffic evenly regardless of state. LLM inference is stateful: each vLLM replica holds a different KV cache. Routing to the wrong replica means throwing away computed work and re-running prefill from scratch. The routing logic must be inference-aware.

The Pipeline

1. **Filter.** Narrows the candidate set before any scoring. Saturated pods are dropped entirely so overloaded replicas don't get more load piled on them, even if they hold a cache-warm option.
2. **Score.** Every survivor gets a composite numeric score from weighted scorers. Two matter most:
   - **Prefix-cache scorer** — Ranks pods by the length of consecutive matching KV cache blocks from the start of the prompt.
   - **Token-load scorer** — Ranks pods by queued prefill work rather than raw request count, since two pods with the same queue depth may have very different actual workloads.
3. **Pick.** The pod with the highest composite score wins. A single destination — never a fan-out.

The KV Cache Indexer

The EPP maintains a global, near-real-time index of KV cache block locality across all vLLM pods. When routing, it converts the prompt's prefix tokens into deterministic block keys matching vLLM's internal logic, queries the index to find which pods hold those blocks, and ranks pods by matching-block count. Common repeated prompts get scored in sub-millisecond time.

Two Scenarios

Situation	Routing outcome
Brand-new conversation	Neither GPU has relevant cache. Prefix-cache scorer contributes nothing; decision collapses to the load scorer alone. Whichever pod has less queued work wins.
Follow-up in ongoing chat	The pod that handled the first turn has the cache warm. Indexer finds a long block match, scores it highest, request goes back to the same pod — avoiding full prefill recomputation. Unless that pod is now saturated, in which case the filter drops it and a fallback fresh prefill happens elsewhere.

08 Disaggregated Serving — P/D as Separate Fleets

> "Prefill-only" and "decode-only" are not just labels on identical pools. They are vLLM instances configured to run only one half of inference for a given request. Each request physically crosses from one to the other mid-flight.

The Four-Stage Flow

1. **Pairing.** The EPP applies its filter/score/pick pipeline separately for prefill and decode, choosing one worker of each type as a matched pair for this request. Requests without disaggregation headers skip this and are served locally on a decoder.

2. **Prefill executes and stops.** The prefill worker runs the full compute-bound forward pass over the entire prompt to build the KV cache, then stops — it never generates real output tokens. It constructs KV transfer parameters (including its pod address) so the decode side knows where to reach it.

3. **KV transfer — the hard engineering.** The decode worker pulls the KV cache directly from the prefill worker's GPU memory via one-sided RDMA reads. GPUDirect RDMA registers GPU memory with the NIC so the transfer bypasses CPU and host memory entirely. NVIDIA NIXL is the transport library making this portable, supporting UCX, RoCE, and InfiniBand. Connections handshake once per pod pair (about 5 seconds) and are reused for every subsequent request.

4. **Decode takes over.** Holding the cache locally, the decode worker runs the autoregressive loop as usual, streaming tokens back as they are produced. The prefill worker is done with this request entirely.

DEVOPS REALITY CHECK

Disaggregation stops being a Kubernetes scheduling problem and becomes a **network topology** problem. Prefill and decode node pools cannot just be "GPU instances in a VPC" — they need RDMA-capable NICs, placement groups keeping P/D pairs physically close, and security-group rules permitting the RDMA transport specifically. Without RDMA, the transport falls back to TCP, which dominates TTFT and is suitable only for testing. This is meaningfully more infrastructure complexity than a standard EKS + ALB setup and is worth flagging as an explicit design trade-off in interviews.

09 The Three Well-Lit Paths

> llm-d's team publishes three tested, benchmarked deployment recipes. They are progressive, not exclusive — a production DeepSeek-R1 deployment realistically runs all three at once.

Path 1 — Optimized Baseline

Identical vLLM pods, cache-aware routing on, no disaggregation, no model slicing. Pure software win with no new node pools, networking, or model changes required.

- **Benchmark:** ~57× faster TTFT and 2× throughput vs. round-robin, measured on 8 pods across 16 H100 GPUs.
- **When to use:** Always. This is the first optimization to evaluate — before adding hardware, tuning parameters, or exploring quantization.

Path 2 — Split the Work (P/D Disaggregation)

Prefill and decode on separately sized pods, connected by RDMA KV transfer.

- **Benchmark:** Up to 50,000 output tokens/second on a 16×16 B200 topology, with order-of-magnitude TTFT reduction. GPT-OSS on NVIDIA B200: up to 70% higher tokens/sec vs. standard vLLM. GPT-OSS-120B and Llama 3.3 70B on AMD MI300X: 10–30% throughput improvement on identical infrastructure.
- **When to use:** Long, variable-length prompts where prefill/decode interference on shared GPUs is significant. Short uniform prompts do not create enough interference to justify the RDMA transfer cost.

Path 3 — Spread the Model (Wide Expert Parallelism)

A single model too large for any one GPU, sliced across many GPUs cooperating on every forward pass. Mixture-of-Experts (MoE) architectures make this tractable: each token routes through only a handful of "expert" sub-networks. Expert Parallelism (EP) places experts on different GPUs; Data Parallelism (DP) scales throughput; together they let giant MoE models be served without any GPU needing the whole thing.

- **Benchmark:** 2,200 tokens/sec per H200 GPU; ~3,100 tokens/sec per B200 GPU. Sustained multi-node throughput of ~50k tokens/sec on 16×16 B200 topology.
- **When to use:** Not optional. Required as soon as the model genuinely cannot fit on one GPU (DeepSeek-R1, GPT-OSS-class models), regardless of traffic pattern.

How They Combine

A production giant-MoE deployment runs all three simultaneously: Path 3's expert-parallel GPU groups *are* the individual "prefill worker" and "decode worker" units from Path 2, and Path 1's cache-aware EPP routing sits on top of everything, deciding which expert-parallel group handles each request.

10 Kubernetes Objects — The Full Stack

> llm-d is a first-class Kubernetes citizen. Every routing decision, every pod, every scaling event is a standard Kubernetes primitive or a well-defined CRD. Nothing runs beside the cluster.

The Two Helm Releases

Release	What it installs
llm-d-router (or gateway)	Gateway resource, HTTPRoute, InferencePool CRD, EPP Deployment + Service, RBAC glue. The *routing plane*. Serves any model.
llm-d-modelservice	The actual model workloads — prefill pods and decode pods as separate Deployments / StatefulSets / LeaderWorkerSets, labeled to match the InferencePool selector.

The split maps to two personas: platform team owns the router (one-time install), model owners deploy and update model services independently. Analogous to installing `nginx-ingress` once and then deploying many application charts behind it.

The Two New CRDs

CRD	Purpose
InferencePool	Defines a pool of model-server pods via label selector. The HTTPRoute's `backendRef`. Owned by the platform-team persona.
InferenceObjective (formerly InferenceModel)	Maps a model name to a backend pool. Handles priority and request criticality. Owned by the model-owner persona.

The Object Chain for One Request

```
Client → Gateway → HTTPRoute → InferencePool → EPP (ext-proc) → vLLM pod
```

Every one of these is inspectable with `kubectl get`. Envoy (backing the Gateway) makes a gRPC ext-proc call out to the EPP pod for every request: *"which backend should this go to?"* — and the EPP answers using its filter/score/pick pipeline.

Choosing the Right Pod Primitive

Primitive	When to use	Why
Deployment	Model fits on one GPU; pods interchangeable.	Default. Decouples replica count from node count.
StatefulSet	Model fits on one GPU; need cached weights across restarts.	Per-pod PersistentVolume avoids re-downloading 140GB Llama-3.1-70B weights on every restart (which takes ~1 hour and wastes GPU dollars). Stable pod identity (`prefill-0`) also helps cache-affinity routing.
LeaderWorkerSet (LWS)	Model spans multiple GPUs on multiple nodes (Path 3).	Treats a group of pods as a single unit of replication. Scaling to 2 replicas of a 4-pod group means 8 pods, scheduled together, restarted together. One shard cannot scale independently — a client request needs all shards cooperating simultaneously.

LeaderWorkerSet in Practice

```yaml
apiVersion: leaderworkerset.x-k8s.io/v1
kind: LeaderWorkerSet
metadata:
  name: prefill-workers
spec:
  replicas: 4                    # 4 prefill groups
  leaderWorkerTemplate:
    size: 2                      # each group is 2 pods (1 leader + 1 worker)
    leaderTemplate:
      spec:
        containers:
        - args: ["--role=prefill", "--tensor-parallel-size=2"]
    workerTemplate:
      spec:
        containers:
        - args: ["--role=prefill"]
```

Prerequisites

- Kubernetes 1.29+
- cert-manager
- Gateway API CRDs v1.3.0+
- Gateway API Inference Extension CRDs
- NVIDIA GPU Operator (device plugin, drivers, DCGM exporter)

- For disaggregation: RDMA-capable node networking (InfiniBand or RoCE) — cloud-provider-specific configuration required
- Prometheus Operator + ServiceMonitor CRD for metrics

11 The Final Answer

What Exactly Happens When Somebody Writes a Prompt and Hits Enter

The complete lifecycle, from keyboard to GPU to your screen — every layer, every object, every second-and-a-half of the pause and every millisecond of the stream.

Stage 0 — Before the Request Leaves

The client (browser, mobile app, CLI, or your CI pipeline calling the Claude API for security-gate triage) serializes the prompt into an HTTPS POST body. Auth tokens are attached. TLS session is established or reused.

Stage 1 — Edge and CDN

The request hits a global edge network — Cloudflare-class or a hyperscaler's own edge. TLS is terminated close to the user for latency reduction. DDoS mitigation and bot detection run inline. Geo-routing sends the request to the nearest regional cluster with GPU capacity.

Stage 2 — API Gateway

Auth token or API key is validated. Per-user and per-org rate limits are enforced — measured in **tokens-per-minute**, not just requests-per-minute, because token count drives compute cost and abuse potential. Tiered access sends free-tier requests to different backends than paid-tier.

Stage 3 — Safety Classification

A small, fast classifier model scans the prompt for policy violations, prompt injection, and jailbreak attempts *before* the expensive main model ever sees it. Conceptually a WAF rule or Kyverno admission policy — cheap to run, rejects bad input early.

Stage 4 — Into the Kubernetes Cluster

The request enters the Gateway resource. Envoy sees the HTTPRoute rule matches and its `backendRef` points to an InferencePool rather than a plain Service. Envoy pauses and makes a gRPC ext-proc call to the EPP pod: *"which pod handles this?"*

Stage 5 — The Scheduler Decides

The EPP runs its pipeline against every vLLM pod matching the InferencePool selector:

1. **Filter** out saturated pods (high queue depth, KV cache full).
2. **Score** survivors by prefix-cache match length and token load.
3. **Pick** one prefill pod and one decode pod as a matched pair.

The EPP returns the prefill pod address to Envoy.

Stage 6 — Prefill (The Pause Begins)

Envoy forwards the request to the chosen prefill pod. The vLLM process:

1. Tokenizes the prompt on CPU (BPE merge rules → integer sequence).
2. Copies tokens across the PCIe bus into GPU VRAM.
3. Runs one compute-bound forward pass over the entire prompt, in parallel — all tokens processed simultaneously.
4. The output of this pass is the full KV cache for the prompt, held in VRAM.
5. The prefill pod signals ready. It generates no real output tokens.

Stage 7 — KV Transfer (Still the Pause)

The paired decode pod pulls the KV cache from the prefill pod's GPU memory using NIXL over RDMA — GPU-to-GPU, bypassing CPU and host memory entirely. Over InfiniBand or RoCE this is fast enough to be hidden behind normal scheduling overhead. Over plain TCP it dominates TTFT.

Stage 8 — Decode (The Stream Begins)

The decode pod now holds the KV cache locally. The autoregressive loop starts:

- One memory-bandwidth-bound forward pass reads the model's weights and the growing KV cache from VRAM.
- It produces exactly one new token.
- That token is appended to the KV cache and streamed back through the Gateway to your screen the instant it exists.
- The loop repeats until an end-of-sequence token, a max-token limit, or the client disconnects.

An output safety classifier runs in parallel on the stream, capable of truncating a response mid-generation if it drifts into disallowed content.

Stage 9 — Response Reaches You

Tokens arrive as server-sent events (or an equivalent streaming protocol), detokenized back into UTF-8 text, and rendered in your interface word by word. This is why you see text appear incrementally — each word really is the output of one discrete, sequential GPU pass, streamed the instant it was produced.

What Runs Off the Main Path

Concurrent with everything above, and not blocking the response:

- **Prometheus scrapes** vLLM's `/metrics` endpoint — TTFT, TPOT, KV cache utilization, queue depth, tokens-per-second.
- **OpenTelemetry traces** emit spans for each layer of the pipeline.
- **Loki collects logs**; Falco runs runtime anomaly detection on the serving containers.
- **Alertmanager routes** any TPOT regression or KV-cache-saturation alert to on-call — potentially through your AI alert triage webhook that calls the Claude API for enrichment.
- **Autoscaling controllers** react to Prometheus custom metrics: HPA or KEDA scales the prefill pool on request rate, the decode pool on KV-cache occupancy.
- **ArgoCD watches** the Git config repo; if a new model version was merged to `prod`, it begins a canary rollout of new pods behind the InferencePool.

> **THE ONE-PARAGRAPH INTERVIEW VERSION**
>
> A prompt travels through edge/CDN and an API gateway that meters tokens-per-minute, is passed through an input-safety classifier, and enters the cluster at a Gateway API Gateway backed by Envoy. Envoy's HTTPRoute targets an InferencePool CRD, and for every request Envoy makes a gRPC ext-proc call to an Endpoint Picker pod that runs a filter/score/pick pipeline over all matching vLLM pods — filtering out saturated ones, scoring survivors by KV-cache prefix match and current token load, and picking one prefill pod and one decode pod as a pair. The prefill pod tokenizes on CPU, moves tokens across PCIe to VRAM, runs one compute-bound forward pass over the entire prompt to build the KV cache, and stops. The decode pod pulls that cache from the prefill pod's GPU memory over RDMA using NIXL — bypassing CPU entirely — then runs the autoregressive decode loop, producing one memory-bandwidth-bound token per forward pass and streaming each back through the Gateway the instant it exists. Prefill pods, decode pods, and the EPP are all deployed via Helm as Deployments, StatefulSets, or LeaderWorkerSets depending on whether the model fits on one GPU or must be sharded across many; Prometheus scrapes vLLM's metrics endpoint for TTFT, TPOT, and KV-cache utilization, which drive HPA/KEDA autoscaling of each pool independently; ArgoCD manages canary rollouts of new model versions from a Git config repo, and Terraform provisions the underlying GPU node pools with RDMA-capable networking. Every component is either a standard Kubernetes primitive, a well-defined CRD, or a Prometheus-scrapable process — which is what makes the whole thing operable with the same DevOps toolchain used for any other workload.

12 The Complete Cloud & DevOps Concept Map

> Every cloud and DevOps concept that lives inside this architecture, mapped to where it actually applies — so you can see exactly which of your existing skills transfers and where the AIDevSecOps-specific gaps sit.

Source Control & CI/CD

Concept	Where it applies in this stack
Git four-branch model (dev → qa → ppd → prod)	Model version promotion. Same model image passes through every environment; only Helm values differ per environment.
GitHub Actions (SHA-pinned)	Building vLLM container images, running Trivy/Cosign/Syft on them, deploying Helm charts to dev.
Pre-commit hooks (Gitleaks, detect-secrets)	Prevent HuggingFace tokens, cloud credentials, and API keys from being committed to model-service repos.
SBOM (Syft)	Track every dependency in vLLM images. Log4Shell-style regulatory requirement extends fully to AI workloads.
Cosign / Sigstore	Sign model container images and (increasingly) model weights themselves. Kyverno policies enforce that only signed images can schedule to GPU nodes.
AI security gate	Claude API triages CI scan results and posts enriched PR comments. Directly analogous to the input-safety classifier in the inference pipeline.

Containers & Orchestration

Concept	Where it applies
Multi-stage Dockerfiles	vLLM images have heavier CUDA runtime requirements than distroless allows; the pattern still applies for stripping build tools.
Digest pinning	Model images (and llm-d component images) pinned to SHA256 digests in Helm values.
Kubernetes Deployment	Default for single-GPU vLLM pods.
StatefulSet	vLLM pods needing per-pod PVC for cached model weights (avoiding hour-long re-downloads on restart).
LeaderWorkerSet	Multi-node model sharding (Path 3). Groups pods as a single unit of replication.
NVIDIA GPU Operator + **device plugin**	Exposes `nvidia.com/gpu` as a schedulable resource. Handles driver installation, DCGM metrics exporter.
Gateway API (Envoy, Istio, kgateway)	Replaces classic Ingress. Standard implementation used by llm-d's router.
Kyverno / OPA admission control	Enforce signed-image-only policy on GPU nodes; enforce resource requests/limits on vLLM pods.
Helm (per-environment values)	Two releases: `llm-d-router` for routing plane, `llm-d-modelservice` for the model itself.
Kustomize	Overlays for per-environment tweaks on top of Helm-rendered manifests.

GitOps & Deployment

Concept	Where it applies
ArgoCD	Watches config repo; canary-rolls out new model versions or Helm chart updates behind the InferencePool.
ArgoCD Image Updater	Automatically bumps the model container image digest when a new version passes CI.
Two-repo GitOps pattern	App-config repo separate from Terraform-infra repo. ArgoCD watches the former; Terraform Cloud/Atlantis handles the latter.
Push-based CD vs. GitOps pull	Same argument applies: no cluster credentials in CI secrets; ArgoCD inside the cluster pulls config from Git.
Canary / progressive delivery	New model versions rolled out to a fraction of the InferencePool first, with automatic rollback if eval scores or latency SLOs regress.

Infrastructure as Code

Concept	Where it applies
Terraform / Terragrunt	EKS cluster, VPC, GPU node groups (separate pools for prefill and decode), RDMA-capable subnets, security groups.
EKS with GPU node pools	Instance types like `p5.48xlarge`, `p6-b200`. Taints ensure only GPU workloads schedule to them.
Placement groups / cluster networking	Keep P/D pairs physically close for RDMA latency.
IRSA / Pod Identity	vLLM pods pull model weights from S3 without long-lived credentials.
Secrets Store CSI Driver	Mount HuggingFace tokens and model access credentials from AWS Secrets Manager.
WAF	Sits in front of the Gateway; adds an inference-agnostic first line of defense.
Multi-account landing zone	Separate accounts for prod/stage/dev GPU clusters, multi-region for capacity and disaster recovery.
ALB Controller	Backs the Gateway resource on EKS.

Observability

Concept	Where it applies
Prometheus + ServiceMonitor	Scrapes vLLM's `/metrics` endpoint. New metric names: `vllm:kv_cache_usage_perc`, `vllm:num_requests_running`, `vllm:time_to_first_token_seconds`, `vllm:time_per_output_token_seconds`.
Grafana	Dashboards for TTFT, TPOT, KV-cache occupancy, tokens-per-second, GPU utilization (via DCGM exporter).
Loki	Aggregated logs from vLLM pods, EPP, gateway.
OpenTelemetry	Distributed tracing across gateway → EPP → prefill → decode. Reveals where latency actually accumulates.
Jaeger	Trace visualization backend for OTel spans.
Falco	Runtime security anomaly detection on inference containers.
Alertmanager	Routes SLO regression alerts. Enriched via AI-triage webhook calling the Claude API.
Custom SLOs	TTFT p95, TPOT p99, KV-cache saturation percentage, tokens-per-second per GPU. Not HTTP-status-code error rates.

Autoscaling

Concept	Where it applies
HPA on custom metrics	Standard pattern, GPU-serving-specific triggers: `kv_cache_usage_perc` for decode, request-rate for prefill.
KEDA	More flexible custom-metric triggers than HPA alone.
Cluster autoscaler / Karpenter	Provision GPU nodes on demand. Note: scale-up latency is much worse than CPU because a new node needs drivers, model weights in VRAM, and warm-up before it's useful.
Gang scheduling (Volcano, KAI Scheduler)	Required for multi-pod LWS groups — all pods in a group must schedule together or none do.

Security & Compliance

Concept	Where it applies
Image scanning (Trivy)	vLLM images before deployment. Extends to model weights in some emerging tooling.
SBOM (Syft)	Full dependency graph of every image running in the inference stack.
Policy-as-code (Checkov)	Terraform IaC scanning — public S3 buckets, unencrypted volumes, overly permissive security groups.
Input safety classifier	The AIDevSecOps analog of a Kyverno admission policy or WAF rule — applied to prompts before they reach expensive compute.
Output safety classifier	Runs on the streaming decode output; can truncate mid-response.
Data residency	Self-hosting vLLM inside VPC boundary for regulated workloads (healthcare, finance, government). Prompts never leave the customer's network.
MIG (Multi-Instance GPU)	Hard tenant isolation on shared GPUs, at the cost of shared-batching throughput.

Networking

Concept	Where it applies
Envoy / ext-proc	Gateway calls out to EPP as a gRPC ext-proc backend for every inference request.
Server-sent events (SSE)	Streaming protocol for token-by-token decode output.
RDMA (InfiniBand, RoCE)	GPU-to-GPU KV cache transfer between prefill and decode pods. Non-standardized across cloud providers.
NIXL	NVIDIA's transport library for KV transfers; supports UCX, RoCE, InfiniBand backends.
GPUDirect RDMA	NIC registers GPU memory directly, bypassing host memory and CPU.
NVLink	Intra-node GPU-to-GPU for model sharding via tensor parallelism.

THE TRANSFERABLE SKILLS SUMMARY

Of the concepts above, the majority are direct one-to-one applications of what a competent DevOps engineer already does. The genuinely new territory is: **GPU-aware scheduling**, **RDMA networking** for disaggregated serving, **LeaderWorkerSet** as a new pod primitive, **vLLM/llm-d** as a new serving-engine class, **token-based rate limiting and cost accounting**, and the **input/output safety classifier** as a new gate category.

Everything else — Helm, ArgoCD, Kyverno, Prometheus, Terraform, IRSA, WAF, SBOM, Cosign, GitOps — transfers unchanged. The AIDevSecOps role is not a different job. It is the same job with a workload that has different resource shapes, different metrics, and one new gate category.